

슈레더 Edge AI 시스템 — 질문 사항 상세 설명

참조 문서: Edge_AI_슈레더_제안서.md 작성일: 2026-03-26

목차

- 각 AI 모델 상세 설명 (10종)
- OPC UA / EtherCAT 통신 방식 설명 및 대안 검토
- 각 센서의 구체적인 제품 모델 추천
- 화재 방지 운영 방식 설명
- Docker 사용 이유와 방법 설명

1. 각 AI 모델 상세 설명

전체 모델 구성 (10종)

영역	#	모델명	알고리즘	실행 주기
고장예지	1	베어링 이상 탐지	LSTM Autoencoder	매 1초
	2	칼날 마모 예측	Gradient Boosting	매 10분
	3	축 불균형 진단	Classical DSP (FFT)	매 1초
	4	이물질 끼임 감지	실시간 패턴 인식	매 10ms
사고예방	5	발화 감지	Rate-of-Change + Anomaly	매 100ms
	6	분진 폭발 예측	CUSUM + 트렌드	매 1초
	7	전해액 누출 감지	Multi-gas Fusion	매 1초
품질관리	8	파쇄 크기 예측	Random Forest	매 1분
	9	RPM 최적화	룰 기반 + 피드백	매 1분
연계분석	10	설비-품질-안전 통합	Ensemble (RF+LSTM+XGB)	매 5분

고장예지 모델 1: 베어링 이상 탐지

개요

항목	내용
목적	축A/B 베어링의 초기 결함을 조기 감지하여 갑작스런 파손 방지
알고리즘	LSTM Autoencoder (Long Short-Term Memory 기반 오토인코더)
입력	VIB-A/B 진동 데이터 → FFT 변환 → 1024개 주파수 bins

항목	내용
출력	이상 점수 (0.0 ~ 1.0)
추론 시간	<10ms
목표 정확도	95%+ (이상 감지율)

LSTM Autoencoder란?

Autoencoder는 "입력을 압축했다가 다시 복원하는" 신경망입니다.

정상 데이터로만 학습:

입력 (1024)
(정상 진동)

인코더

잠재공간 (32)
(압축)

디코더

출력 (1024)
(복원)

정상일 때: 입력 ≈ 출력 → 재구성 오차 "작음" → 이상 점수 낮음
이상일 때: 입력 ≠ 출력 → 재구성 오차 "큼" → 이상 점수 높음

LSTM은 시계열(시간 순서) 데이터의 패턴을 기억하는 특수 신경망입니다. 단순한 Autoencoder보다 진동 데이터처럼 시간에 따라 변하는 신호를 더 잘 학습합니다.

왜 이 알고리즘인가?

장점	설명
비지도 학습	고장 데이터가 없어도 "정상 패턴"만으로 학습 가능 (실제 베어링 고장 데이터는 매우 드물기 때문)
시계열 특화	LSTM이 진동 패턴의 시간적 변화를 포착
미지의 고장 감지	사전에 정의하지 않은 새로운 유형의 이상도 감지 가능
점진적 악화 추적	이상 점수가 서서히 올라가는 것으로 악화 추세 파악

동작 과정 상세

- 1. 데이터 수집: VIB-A, VIB-B (3축 가속도계)에서 100kHz로 원시 진동 데이터 수집
- 2. FFT 변환: 시간 영역 → 주파수 영역으로 변환 (1024개 주파수 bins)
- 3. Envelope Analysis: 베어링 결함 특성 주파수(BPFO, BPFI, BSF, FTF) 추출
 - BPFO (Ball Pass Frequency Outer): 외륜 결함 주파수
 - BPFI (Ball Pass Frequency Inner): 내륜 결함 주파수
 - BSF (Ball Spin Frequency): 전동체 결함 주파수
 - FTF (Fundamental Train Frequency): 보유기 결함 주파수
- 4. LSTM Autoencoder 추론: 현재 스펙트럼을 입력하여 재구성 오차 계산

5. 이상 점수 판정: 오차 크기 → 0~1 사이 정규화 → 등급 분류

학습 방법

단계	내용	소요
1	정상 운전 중 28일간 진동 데이터 수집	28일
2	FFT+Envelope 특징 추출	1일
3	LSTM Autoencoder 학습 (정상 데이터만)	2~4시간 (GPU 서버)
4	검증: 의도적 이상 상태 시뮬레이션 또는 Transfer Learning	1~2일
5	Edge로 TensorRT 변환·배포	1일

고장예지 모델 2: 칼날 마모 예측

개요

항목	내용
목적	칼날의 현재 마모율(%)과 잔여수명(RUL)을 예측
알고리즘	Gradient Boosting (XGBoost 또는 LightGBM)
입력	CUR-A/B, SPD-A/B, VIB-A/B에서 추출한 24개 특징
출력	마모율 (0~100%), 잔여수명 RUL (시간)
추론 시간	<5ms
목표 정확도	$R^2 > 0.92$, RUL 오차 $\pm 15\%$

Gradient Boosting이란?

Gradient Boosting은 "약한 예측 모델 여러 개를 순서대로 합쳐서 강한 모델을 만드는" 기법입니다.

- 1단계 모델: "전류가 높으면 마모율 높다" → 오차 20%
↓ 오차를 줄이기 위해
- 2단계 모델: "진동도 고려하면 더 정확" → 오차 12%
↓ 또 오차를 줄이기 위해
- 3단계 모델: "속도 변동도 반영" → 오차 7%
↓ ... 반복 (100~500단계)
- 최종 모델: 오차 3% → 정확한 마모율 예측

왜 이 알고리즘인가?

장점	설명
정형 데이터 최강	센서 특징값(숫자 표) 예측에서 딥러닝보다 우수

장점	설명
특징 중요도	어떤 센서가 마모 예측에 가장 중요한지 자동 파악
빠른 학습	수분~수십 분 내 학습 완료 (딥러닝은 수시간)
적은 데이터	교체 3사이클(~1,500시간) 데이터면 충분
해석 가능	왜 그런 예측을 했는지 설명 가능 (SHAP 분석)

24개 입력 특징 상세

#	특징명	센서	추출 방법	물리적 의미
1~2	CUR-A/B 평균	CUR	10분 이동평균	마모↑ → 부하↑ → 전류↑
3~4	CUR-A/B 표준편차	CUR	10분 윈도우	마모↑ → 불균일↑ → 편차↑
5~6	CUR-A/B Crest Factor	CUR	피크/RMS	칼날 상태
7~8	CUR-A/B 피크 전류	CUR	10분 최대값	이물질·과부하
9~10	SPD-A/B 평균	SPD	실시간	마모 → 슬립 → 속도 변동
11~12	SPD-A/B 변동률 (COV)	SPD	표준편차/평균	마모 진행도
13~14	VIB-A/B RMS	VIB	10분 RMS	마모↑ → 진동↑
15~16	VIB-A/B Peak	VIB	10분 최대값	충격 성분
17~18	VIB-A/B 2X/1X 비율	VIB	고조파 분석	칼날 불균형
19~20	VIB-A/B 3X/1X 비율	VIB	고조파 분석	칼날 상태
21	누적 운전 시간	카운터	적산	칼날 사용 시간
22	누적 처리량	SCL-1	적산	총 처리 질량
23	현재 RPM 설정	SPD	실시간	운전 조건
24	투입 재료 종류	입력	범주형	NMC/LFP 등

고장예지 모델 3: 축 불균형 진단

개요

항목	내용
목적	축의 불균형(칼날 탈락, 이물질 부착 등)을 실시간 감지
알고리즘	Classical DSP — FFT 1X 성분 분석
입력	VIB-A/B의 1X 회전 주파수 성분
출력	불균형 레벨 (mm/s), 정상/경고/위험 등급
추론 시간	<1ms

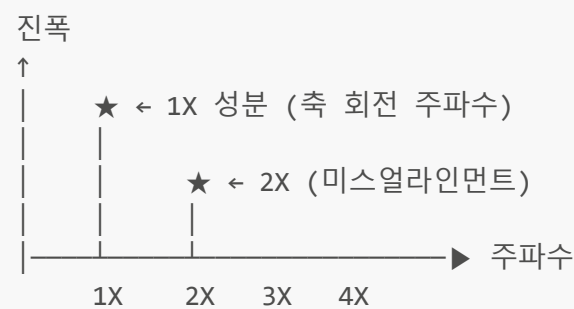
항목	내용
목표 정확도	98%+

원리 설명

축이 회전할 때, 무게가 한쪽으로 쏠려 있으면 **1회전당 1번 진동**이 발생합니다. 이것을 **1X 성분**이라 합니다.

정상 축: 칼날이 균등 → 1X 진동 "작음" (0.5 mm/s)
불균형 축: 칼날 탈락 → 1X 진동 "큼" (5.0 mm/s)

FFT로 분석하면:



- 1X 증가 → 불균형 (칼날 탈락, 이물질 부착)
- 2X 증가 → 미스얼라인먼트 (축 정렬 불량)
- 고주파 증가 → 베어링 결함

왜 Classical DSP인가? (AI가 아닌 이유)

이유	설명
물리 법칙 기반	불균형 → 1X 진동은 물리적으로 확실한 관계. AI 학습이 불필요
극도로 빠름	FFT 1회 연산 <1ms. AI 모델 추론보다 수십 배 빠름
100% 해석 가능	"1X가 5mm/s" = "불균형 있음" — 블랙박스 없음
데이터 불필요	학습 데이터 없이 즉시 적용 가능

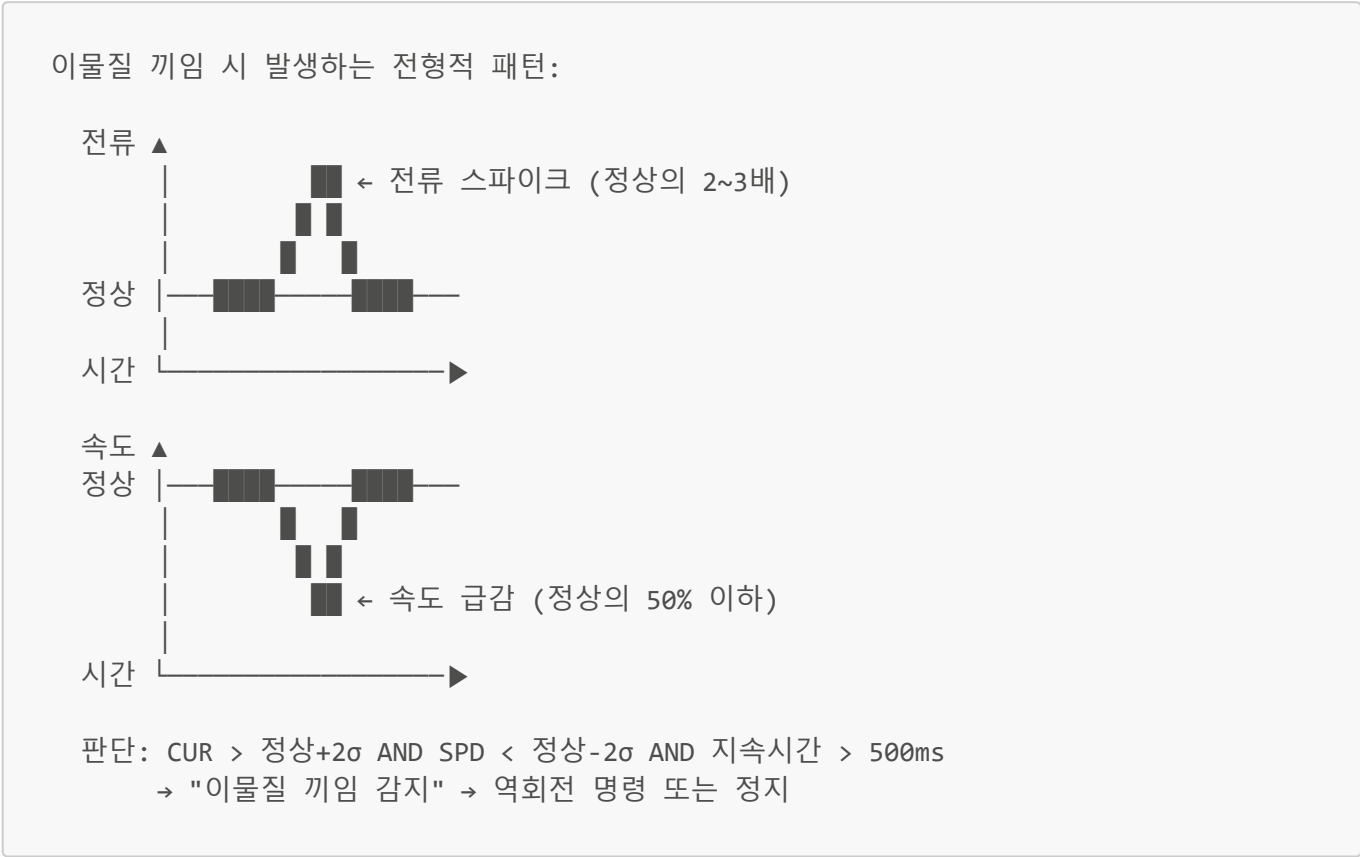
고장예지 모델 4: 이물질 끼임 감지

개요

항목	내용
목적	파쇄 불가능한 이물질이 칼날 사이에 끼었을 때 즉시 감지
알고리즘	실시간 패턴 인식 (Rule-based + Statistical)
입력	CUR-A/B (10ms 간격), SPD-A/B (10ms 간격)

항목	내용
출력	끼임 확률 (0~1), 권고 조치 (역회전/정지)
추론 시간	<5ms
목표 정확도	95%+

감지 로직



왜 Rule-based인가?

이물질 끼임은 패턴이 매우 명확합니다 (전류 급등 + 속도 급감이 동시에 발생). 단순한 룰로도 95% 이상 정확하게 감지되므로, 복잡한 AI 모델이 불필요합니다. 오히려 룰 기반이 더 빠르고 더 신뢰할 수 있습니다.

사고예방 모델 5: 발화 감지

개요

항목	내용
목적	배터리 열폭주/발화를 100ms 이내에 감지하여 자동 대응
알고리즘	Rate-of-Change (변화율 분석) + Anomaly Detection
입력	TMP-IR, TMP-IR2 (이중화), TMP-A/B
출력	발화 확률 (0~1), 발화 추정 위치

항목

내용

추론 시간

<100ms

3중 판단 로직

판단	조건	응답 시간	설명
1차	dT/dt > 5°C/s	<50ms	온도가 초당 5도 이상 급상승 시
2차	T > 150°C	<50ms	절대 온도가 150도 초과 시
3차	IR↑ + GAS↑ + VIB 변화	<100ms	복합 센서 Fusion (AI 판단)

• 1차·2차는 단순 비교 연산 → 극도로 빠름 (<50ms)

• 3차는 AI Fusion Model → 거짓 경보 최소화

Fusion Model 설명

복합 이상 판단은 여러 센서 신호를 종합하여 "정말 발화인지" 판단합니다:

TMP-IR 상승 + GAS-1 VOC 상승 + VIB 패턴 변화
→ 3가지 모두 이상 → 발화 확률 95% → 즉시 대응

TMP-IR 상승 + GAS-1 정상 + VIB 정상
→ 온도만 이상 → 센서 오작동 가능성 → 경고만 발령

이런 복합 판단이 "거짓 경보"를 줄이는 핵심입니다.

사고예방 모델 6: 분진 폭발 예측

개요

항목	내용
목적	분진 농도 추세를 분석하여 폭발 위험 수준 도달 전 사전 경고
알고리즘	CUSUM (누적합 관리도) + 트렌드 예측
입력	DST-1 (레이저 분진 센서) 시계열 데이터
출력	폭발 위험도 (0~1), 예상 위험 도달 시간
추론 시간	<500ms

CUSUM이란?

CUSUM (Cumulative Sum) = 기준값으로부터 벗어난 양을 **누적**하여 합산하는 방법입니다.

일반 임계값 방식:
_____ 임계값 _____
| ← 순간 스파이크는 감지하지만
| 서서히 올라가는 것은 못 잡음

CUSUM 방식:
_____ 임계값 _____
/ ← 서서히 누적되어
/ 결국 임계값 도달
/ → "느린 악화" 감지!

분진 농도가 **서서히** 위험 수준으로 올라가는 것을 일반 임계값 방식으로는 감지하기 어렵습니다. CUSUM은 작은 변화가 누적되는 것을 감지하여 **폭발 위험에 도달하기 전에** 경고합니다.

사고예방 모델 7: 전해액 누출 감지

개요

항목	내용
목적	배터리 전해액 누출을 가스 패턴으로 감지
알고리즘	Multi-gas Fusion (다중 가스 복합 분석)
입력	GAS-1 복합가스센서 (VOC, H ₂ , CO 동시 측정)
출력	누출 확률 (0~1), 추정 가스 종류
추론 시간	<1s

Multi-gas Fusion이란?

전해액이 분해되면 여러 가스가 **동시에** 발생합니다:

가스	정상 시	전해액 누출 시	다른 원인일 때
VOC (유기 화합물)	낮음	높음	도장/세정제도 높을 수 있음
H ₂ (수소)	없음	중간~높음	배터리 가스 발생 시
CO (일산화탄소)	없음	낮음~중간	연소 반응 시

단일 가스만 보면 오판이 잦지만, **3가지를 동시에 분석**하면:

VOC↑ + H₂↑ + CO↑ → 전해액 누출 확률 95% → 경보
VOC↑ + H₂= + CO= → 환경 오염 (세정제 등) → 무시
VOC= + H₂↑ + CO= → 배터리 가스 발생 → 주의
VOC= + H₂= + CO↑ → 외부 연소 가스 → 환기 확인

품질관리 모델 8: 파쇄 크기 예측

개요

항목	내용
목적	센서 데이터로 파쇄물의 예상 크기를 간접 추정
알고리즘	Random Forest (랜덤 포레스트)
입력	CUR, SPD, VIB, SCL 등 12개 특징
출력	예상 파쇄 크기 (mm), 균일도 (%)
추론 시간	<5ms
목표 정확도	$R^2 > 0.92$

Random Forest란?

"의사결정나무(Decision Tree) 수백 개를 만들어서 다수결로 예측하는" 방법입니다.

나무 1: "전류 높고 RPM 빠르면 → 작게 파쇄 (65mm)"
나무 2: "처리량 적고 RPM 빠르면 → 작게 파쇄 (60mm)"
나무 3: "전류 높고 진동 크면 → 중간 (75mm)"
... (200개 나무)

최종 예측 = 200개 나무의 평균 = 68mm

왜 간접 측정인가?

슈레더 배출부는:

- 고온 (N₂ 분위기, 밀폐)
- 분진 (카메라/레이저 장비 오염)
- 충격 (대형 파쇄물 낙하)

→ 인라인 입도 분석기 설치가 물리적으로 매우 어려움 → 전류/속도/진동 등 이미 설치된 센서로 간접 추정하는 것이 현실적

품질관리 모델 9: RPM 최적화

개요

항목	내용
목적	투입 재료와 현재 상태에 맞는 최적 RPM을 추천
알고리즘	룰 기반 (Look-up Table) + PID 피드백

항목	내용
입력	파쇄 크기 예측값 + 목표값 + 재료 종류
출력	축A/B 최적 RPM
추론 시간	<100ms

동작 방식

1단계: 초기 RPM 결정 (Look-up Table)

배터리 종류	크기	권장 RPM
NMC 소형	소	20 RPM
NMC 대형	대	15 RPM
LFP 소형	소	22 RPM
LFP 대형	대	18 RPM

2단계: 피드백 조정 (PID 제어)

AI가 예측한 파쇄 크기가 목표보다 크면 → RPM +1~2

AI가 예측한 파쇄 크기가 목표보다 작으면 → RPM -1~2

→ 실시간으로 미세 조정

왜 AI가 아닌 룰 기반인가?

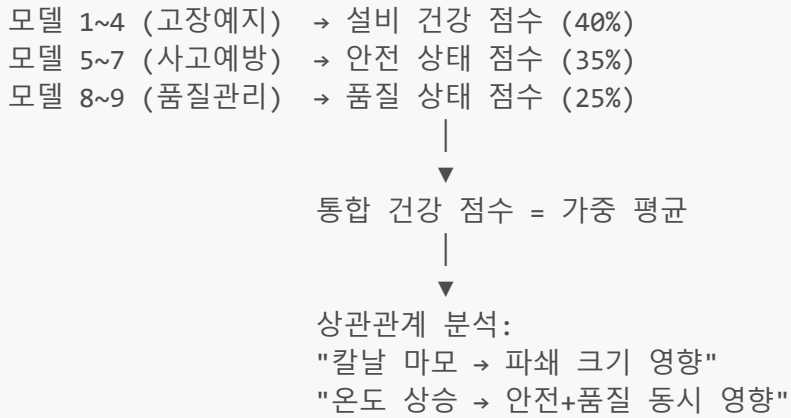
- RPM 조정은 **안전에 직결되는** 제어 → 예측 가능하고 해석 가능한 방법이 더 안전
- 제어 변수가 RPM 하나 → 복잡한 AI보다 PID 피드백이 더 안정적
- 현장 운전자가 **왜 그 RPM인지 즉시 이해** 가능

연계분석 모델 10: 설비-품질-안전 통합

개요

항목	내용
목적	설비 상태 변화가 품질과 안전에 미치는 영향을 종합 분석
알고리즘	Ensemble (Random Forest + LSTM + XGBoost 결합)
입력	모든 센서 데이터 + 모델 1~9의 출력값
출력	통합 건강 점수 (0~100), 상관관계 맵
추론 시간	<100ms

동작 방식



이 모델은 다른 모델들의 결과를 입력으로 사용하므로, 개별 모델로는 보이지 않는 **영역 간 상관관계**를 발견할 수 있습니다.

2. OPC UA / EtherCAT 통신 방식

2.1 OPC UA란?

항목	내용
정식명	Open Platform Communications Unified Architecture
목적	산업 자동화 장비 간의 표준화된 데이터 교환
특징	플랫폼·벤더 독립, 보안 내장 (암호화+인증), 데이터 모델링
통신 속도	일반적으로 10~100ms 주기 (Ethernet 기반)
사용 예시	Edge AI ↔ PLC 데이터 공유, 센서 ↔ Edge 통신

쉬운 비유

OPC UA는 ****산업계의 "공통 언어"****입니다. 마치 전 세계 사람들이 영어로 소통하듯, 서로 다른 제조사의 장비들이 OPC UA로 데이터를 주고받습니다.

기존 (OPC UA 이전):

Siemens PLC ← 전용 프로토콜 → Siemens HMI만 가능
 Mitsubishi PLC ← 다른 프로토콜 → Mitsubishi HMI만 가능
 → 서로 다른 제조사 장비끼리 통신 어려움

OPC UA 적용 후:

Siemens PLC ┌
 Mitsubishi PLC ┤ — OPC UA → Edge AI (어떤 PLC든 연결 가능)
 Allen-Bradley └

2.2 EtherCAT이란?

항목	내용
정식명	Ethernet for Control Automation Technology
목적	산업용 실시간 이더넷 통신 (필드버스)
특징	극도로 빠른 주기 (125μs~1ms), 마스터-슬레이브 구조, 확정적 시간 보장
통신 속도	100 Mbps, 1000개 I/O를 30μs에 처리 가능
사용 예시	고속 DAQ(데이터 수집) 모듈 ↔ Edge AI

쉬운 비유

EtherCAT은 *****열차 배달 시스템*****입니다. 데이터 패킷이 기차처럼 각 장치를 순서대로 통과하면서 데이터를 싣고 내립니다.

일반 Ethernet:

마스터 → 장치1 → 마스터 → 장치2 → 마스터 → 장치3
(매번 왕복 → 느림)

EtherCAT:

마스터 → 장치1 → 장치2 → 장치3 → 마스터 (한 바퀴에 다 처리)
(한 번에 모든 장치 처리 → 극도로 빠름)

2.3 이렇게 빠른 통신이 필요한가? 검토

통신 대상	필요 속도	OPC UA 적합?	EtherCAT 필요?
진동 센서 (VIB)	100kHz 연속	× 느림	× 별도 DAQ 보드 사용
Safety PLC ↔ Edge AI	10~100ms	적합	불필요
발화 감지 온도 (IR)	100ms	적합	불필요
전류/속도/분진/가스	1~10ms	적합	불필요
Edge AI ↔ Cloud	1~10s	MQTT 적합	불필요

결론

슈레더 단독 적용에서는 **EtherCAT**이 반드시 필요하지 않습니다.

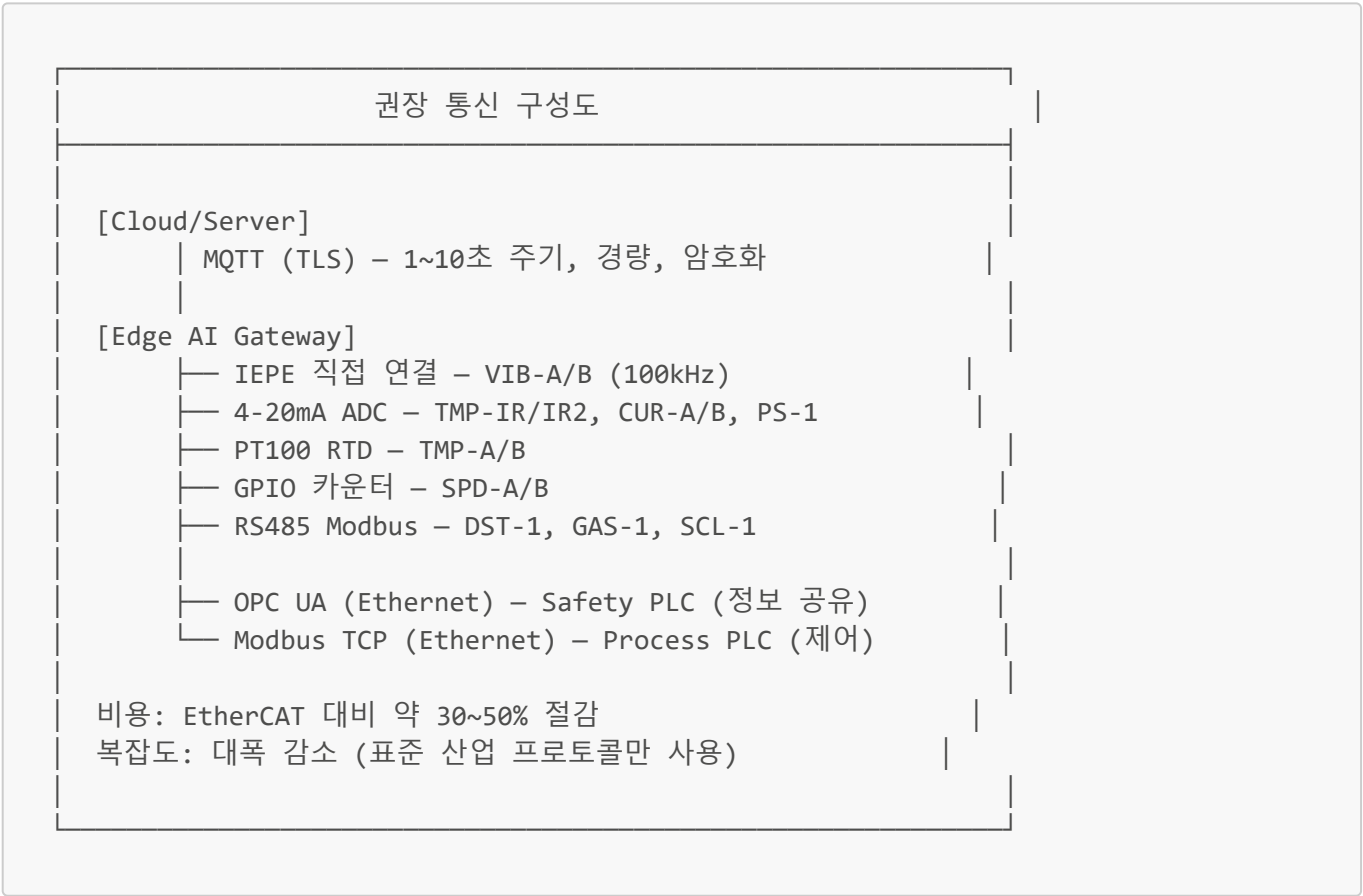
- 진동 센서: 전용 **고속 DAQ 보드** (NI/Beckhoff 등)를 Edge에 직접 연결 → EtherCAT 불필요
- PLC 통신: **OPC UA**로 충분 (10~100ms 주기면 안전 판단에 충분)
- 클라우드: **MQTT**로 충분 (수초 주기)

2.4 실용적 대안 추천

통신 구간	권장 방식	속도	비용	이유
-------	-------	----	----	----

통신 구간	권장 방식	속도	비용	이유
진동 센서 → Edge	전용 DAQ (IEPE 직접 연결)	100kHz+	중	아날로그 직접 수집이 가장 빠르고 정확
아날로그 센서 → Edge	4-20mA → ADC 모듈	1kHz	저	산업 표준, 배선 간단
디지털 센서 → Edge	RS485 Modbus RTU	9600~115200 bps	저	저비용, 분진/가스 센서 표준
Edge → Safety PLC	OPC UA (Ethernet)	10~100ms	저	표준, 보안 내장, 양방향
Edge → Process PLC	Modbus TCP	10~100ms	저	간단, 대부분 PLC 지원
Edge → Cloud	MQTT (TLS)	1~10s	저	IoT 표준, 경량, 암호화
현장 HMI ← Edge	HTTP/WebSocket	100ms	저	웹 기반 대시보드

최종 권장 구성



3. 센서 구체 제품 모델

3.1 제품 추천 총괄표

#	센서 ID	용도	추천 제품	제조사	주요 사양	예상 단가
1~2	VIB-A/B	3축 진동 가속도계	IMI 603C91	PCB/IMI Sensors	100mV/g, IP67, IEPE	\$800~1,200
3~4	TMP-A/B	PT100 온도 센서	WIKA TR10-C	WIKA	Class A, -50~400°C, 3-wire	\$80~150
5,10	TMP-IR/IR2	적외선 비접촉 온도	Optris CTfast LT	Optris (독일)	-50~975°C, 응답 6ms	\$400~700
6~7	CUR-A/B	CT 전류 센서	LEM AP 200 B420L	LEM (스위스)	0~200A, 4-20mA, True RMS	\$300~500
8~9	SPD-A/B	엔코더	Baumer HOG 10 DN 1024	Baumer (스위스)	1024 PPR, IP67, Heavy Duty	\$500~1,200
11	DST-1	레이저 분진 센서	SICK DUSTHUNTER SB50	SICK (독일)	후방산란식, 단면 설치	\$3,000~5,000
12	GAS-1	복합가스 센서	Dräger Polytron 8100 (H ₂) + Polytron 7000 (VOC)	Dräger (독일)	H ₂ /VOC/CO 개별	\$1,500~3,500 각
13	PS-1	초음파 근접 센서	Pepperl+Fuchs UC500-30GM	P+F (독일)	30~500mm, 4-20mA	\$300~500
14	SCL-1	컨베이어 스케일	Siemens Milltronics MUS	Siemens	±0.5% 정확도, 모듈형	\$3,500~7,000

3.2 각 센서 상세

VIB-A/B: IMI 603C91 (3축 진동 가속도계)

항목	사양
감도	100 mV/g
주파수 범위	0.5 Hz ~ 10 kHz
측정 범위	±50g
출력	IEPE (ICP)
방진	IP67
설치	Ring-style 마운트 (볼트 고정)
크기	직경 25mm × 높이 30mm
동작 온도	-50~121°C

대안: PCB 356A16 (triaxial, 산업 표준), Brüel & Kjær 4524-B (고정밀)

TMP-IR/IR2: Optris CTfast LT (적외선 비접촉 온도)

항목	사양
측정 범위	-50 ~ 975°C
응답 시간	6ms (발화 감지에 필수적인 초고속 응답)
광학 해상도	15:1
출력	4-20mA, 0-10V, RS485
방진	IP65 (에어 퍼지 옵션 시 분진 환경 가능)
스펙트럼	8-14μm

대안: Micro-Epsilon CTM-3 (고정밀), Raytek MI3 (소형, Modbus)

선정 이유: 발화 감지에서 가장 중요한 것은 **응답 시간**입니다. Optris CTfast LT의 6ms 응답은 업계 최고 수준으로, 배터리 열폭주의 급격한 온도 상승을 놓치지 않습니다.

GAS-1: Dräger Polytron 시리즈 (복합가스)

배터리 전해액 감지에는 여러 가스를 동시에 측정해야 합니다. Dräger의 Polytron 시리즈를 조합합니다:

가스	모델	센서 기술	측정 범위
H ₂ (수소)	Polytron 8100 EC	전기화학	0-1000 ppm
VOC (유기화합물)	Polytron 7000	PID (광이온화)	0-2000 ppm
CO (일산화탄소)	Polytron 8100 EC	전기화학	0-500 ppm

대안 (올인원): Honeywell Searchpoint Optima Plus (적외선, 다중가스), MSA Ultima X5000 (4채널)

3.3 국내 구매처 참고

브랜드	국내 대리점	연락처
IMI/PCB Sensors	한국PCB (주)	www.pcbkorea.co.kr
WIKA	위카코리아	www.wika.co.kr
Optris	이레테크	www.optris.co.kr
LEM	한국LEM	www.lem.com
Baumer	바우머코리아	www.baumer.com/kr
SICK	지크코리아	www.sick.com/kr
Dräger	드레거코리아	www.draeger.com/ko_kr
Pepperl+Fuchs	P+F코리아	www.pepperl-fuchs.com/korea
Siemens	한국지멘스	www.siemens.co.kr

4. 화재 방지 운영 방식

4.1 배터리 파쇄 시 화재가 발생하는 이유

단계	현상	원인
1	파쇄 중 셀 손상	칼날이 배터리 셀의 양극-음극 사이 분리막을 파괴
2	내부 단락 발생	양극과 음극이 직접 접촉하여 전류가 흐름
3	열 발생 (Joule heating)	단락 전류에 의한 급격한 온도 상승
4	전해액 기화	온도 상승으로 인화성 전해액이 기화
5	열폭주 (Thermal Runaway)	양극 물질 분해 → 산소 자체 생성 → 가속 연쇄 반응
6	발화/폭발	기화된 전해액 + 산소(자체 생성) → 화재 또는 폭발

핵심 위험: 리튬이온 배터리는 자체적으로 산소를 발생시키므로, 외부 산소를 차단해도 완전히 안전하지 않습니다. 다만 N₂ 분위기는 전해액 증기의 발화를 크게 억제합니다.

4.2 화재 방지 운영 체계 (5단계)

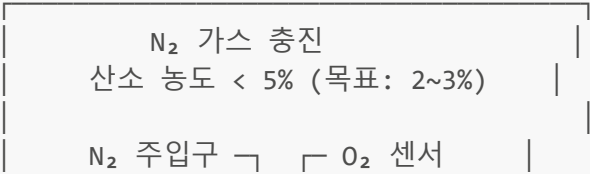
1단계: 사전 방전 (Pre-discharge)

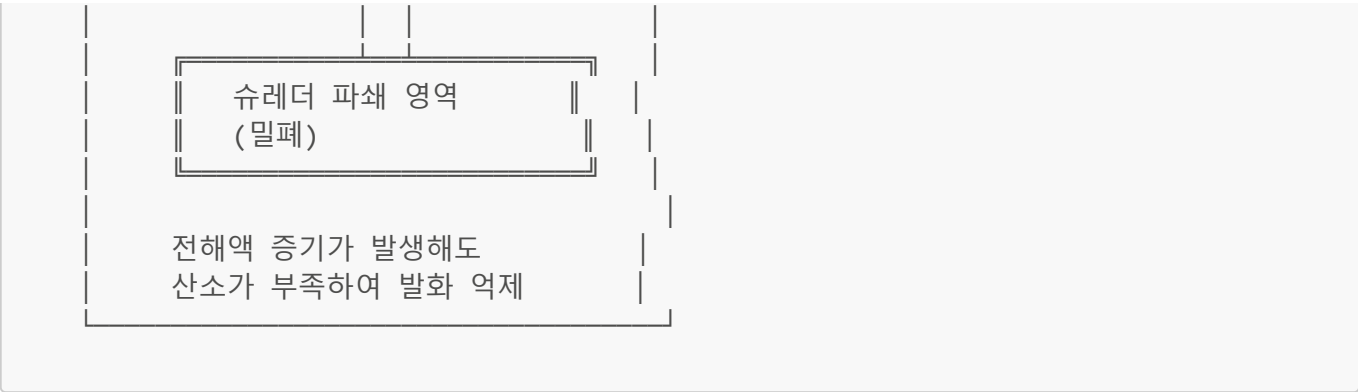
항목	내용
목적	배터리의 잔류 전기 에너지를 제거하여 내부 단락 시 발열 최소화
방법	방전 저항기에 연결하여 전압을 안전 수준(2V 이하)으로 낮춤
대안	소금물 침지 방전, 저온 냉각 방전
효과	잔류 에너지 제거 → 파쇄 시 발열 80% 감소

2단계: N₂ 불활성 분위기 (Nitrogen Inerting)

항목	내용
목적	파쇄 챔버 내 산소 농도를 낮춰 전해액 증기의 발화를 방지
방법	질소(N ₂) 가스를 지속적으로 주입하여 산소 농도 5% 이하 유지
감시	O ₂ 농도센서로 실시간 모니터링 (5% 초과 시 파쇄 중단)
소비량	약 50~200 Nm ³ /h (공정 규모에 따라)

파쇄 챔버 내부:





3단계: 밀폐 + 게이트 밸브 (Sealed System)

항목	내용
목적	외부 공기 유입 차단 + 화재 발생 시 확산 방지
투입구	이중 게이트 밸브 (에어록 방식) — 한쪽이 열리면 다른 쪽은 닫힘
배출구	게이트 밸브 — 화재 시 즉시 폐쇄하여 격리
밀폐성	N ₂ 분위기 유지를 위해 챔버 전체 기밀 설계

투입구 에어록 동작:

상태 1: 외부 밸브 열림, 내부 밸브 닫힘
→ 재료 투입 (N₂ 유출 최소화)

상태 2: 외부 밸브 닫힘, 내부 밸브 열림
→ 재료가 파쇄실로 이동

→ 외부 공기가 파쇄실에 직접 유입되지 않음

4단계: 실시간 감시 + AI 예측

감시 항목	센서	AI 역할
챔버 온도	TMP-IR, TMP-IR2 (이중화)	온도 급변 감지 (<50ms)
가스 농도	GAS-1 (VOC/H ₂ /CO)	전해액 누출 패턴 분석
분진 농도	DST-1	폭발 한계 접근 예측
O ₂ 농도	(별도 센서)	N ₂ 퍼지 상태 확인
베어링 온도	TMP-A, TMP-B	과열로 인한 2차 발화 방지

AI의 차별점: 단순 임계값이 아닌, 복합 패턴 분석으로 거짓 경보를 줄이고 진짜 위험을 더 빨리 감지

5단계: 자동 소화 + 격리

화재가 감지되면 자동으로 다음 순서가 실행됩니다:

순서	조치	시간	실행 주체
1	게이트 밸브 폐쇄 (투입/배출 모두)	<500ms	Safety PLC
2	N ₂ 퍼지 최대 출력 (산소 완전 차단)	<1s	Safety PLC
3	자동 소화 시스템 작동 (CO ₂ 또는 분말)	<1s	Safety PLC
4	슈레더 모터 비상 정지	<1s	Safety PLC
5	경보: 사이렌 + 전화 + 비상 방송	<3s	Edge AI + PLC

4.3 소화 시스템 종류

소화 방식	적합도	장점	단점
CO ₂ 소화	★★★	잔여물 없음, 전기 안전	질식 위험 (무인 환경 필수)
분말 소화	★★	빠른 진화, 저렴	잔여물 청소 필요
에어로졸 소화	★★★	소형, 잔여물 적음, 밀폐 공간 적합	고가
물 분무	★	냉각 효과 큼	리튬과 물 반응 위험 (건식 공정에 부적합)

권장: 밀폐 챔버 + N₂ 분위기에서는 **CO₂ 소화** 또는 **에어로졸 소화**가 적합

5. Docker 사용 이유와 방법

5.1 Docker란?

쉬운 비유

Docker는 *****앱을 상자에 담아서 배송하는 시스템*****입니다.

기존 방식 (Docker 없이):

Edge AI 장비

Python 3.8 + TensorRT + NumPy + SciPy + Grafana + TimescaleDB + ZeroMQ + ... 수십 개 라이브러리

→ 하나를 업데이트하면 다른 것이 깨짐
→ 새 장비에 설치하려면 처음부터 반복
→ "내 PC에서는 되는데..."

Docker 방식:

Edge AI 장비

컨테이너1컨테이너2



정확한 정의

Docker는 애플리케이션과 그 실행에 필요한 모든 것(라이브러리, 설정, 종속성)을 ****컨테이너(Container)****라는 격리된 환경에 패키징하는 기술입니다.

5.2 왜 Edge AI 시스템에서 Docker를 사용하는가?

이유	설명	슈레더 시스템 적용
1. 격리	각 AI 모델이 서로 다른 라이브러리 버전을 사용해도 충돌 없음	고장예지(TensorRT) + 품질(scikit-learn) + DB(TimescaleDB) 각각 독립
2. 이식성	개발 PC에서 만든 것이 Edge 장비에서 그대로 동작	GPU 서버에서 학습한 모델을 Jetson Orin에 그대로 배포
3. 업데이트 용이	AI 모델만 교체하고 나머지는 그대로 유지	칼날 마모 모델 v2 배포 시 다른 모델에 영향 없음
4. 롤백	문제 시 이전 버전으로 즉시 되돌림	새 모델이 정확도 떨어지면 1분 내 이전 버전 복구
5. 재현성	동일한 환경을 100% 동일하게 재현	공장 2호기에도 동일 시스템 즉시 복제
6. 안전성	한 컨테이너가 오류나도 다른 컨테이너는 정상 작동	AI 추론 오류 시에도 데이터 수집은 계속됨

5.3 슈레더 Edge AI에서의 Docker 컨테이너 구성

컨테이너	역할	이미지 기반	리소스
ai-pdm	고장예지 AI 추론 (모델 1~4)	NVIDIA L4T + TensorRT + Python	GPU + 2GB RAM
ai-safety	사고예방 AI 추론 (모델 5~7)	NVIDIA L4T + TensorRT + Python	GPU + 1GB RAM
ai-quality	품질관리 AI 추론 (모델 8~9)	Python + scikit-learn	CPU + 512MB RAM
ai-fusion	연계분석 (모델 10)	Python + XGBoost	CPU + 512MB RAM
data-acq	데이터 수집 + 신호처리	Python + NumPy + SciPy	CPU + 1GB RAM
timescaledb	시계열 데이터베이스	TimescaleDB (PostgreSQL)	CPU + 2GB RAM
mqtt-broker	MQTT 메시지 브로커	Mosquitto	CPU + 128MB RAM

컨테이너	역할	이미지 기반	리소스
hmi-server	현장 HMI 웹 서버	Nginx + React	CPU + 256MB RAM

5.4 Docker 기본 사용 방법

Docker 핵심 개념 3가지

개념	비유	설명
Image (이미지)	요리 레시피	컨테이너를 만들기 위한 "설계도". 읽기 전용
Container (컨테이너)	완성된 요리	이미지를 실행한 "실제 동작하는 인스턴스"
Dockerfile	레시피 작성법	이미지를 만드는 "스크립트"

실제 사용 예시: 고장예지 AI 컨테이너

1) Dockerfile 작성 (이미지 설계도)

```
# NVIDIA Jetson용 기본 이미지 사용
FROM nvcr.io/nvidia/l4t-tensorrt:r8.6.2-runtime

# 필요한 Python 패키지 설치
RUN pip install numpy scipy scikit-learn onnxruntime-gpu zmq

# AI 모델 파일 복사
COPY models/ /app/models/
COPY src/ /app/src/

# 실행 명령
CMD ["python", "/app/src/pdm_service.py"]
```

2) 이미지 빌드 (설계도 → 상자 제작)

```
docker build -t shredder-ai-pdm:v1.0 .
```

3) 컨테이너 실행 (상자 열기)

```
docker run -d \
  --name ai-pdm \
  --runtime nvidia \
  --restart always \
  -v /data/sensors:/data \
  shredder-ai-pdm:v1.0
# GPU 사용
# 오류 시 자동 재시작
# 센서 데이터 폴더 연결
```

4) AI 모델 업데이트 (새 모델 배포)

```
# 새 이미지 빌드
docker build -t shredder-ai-pdm:v2.0 .

# 기존 컨테이너 교체 (무중단)
docker stop ai-pdm
docker rm ai-pdm
docker run -d --name ai-pdm --runtime nvidia shredder-ai-pdm:v2.0
```

5) 문제 시 롤백 (이전 버전 복구)

```
# v2.0에 문제가 있으면 v1.0으로 즉시 복구
docker stop ai-pdm
docker rm ai-pdm
docker run -d --name ai-pdm --runtime nvidia shredder-ai-pdm:v1.0
# → 1분 이내 원상 복구
```

Docker Compose로 전체 시스템 관리

실제로는 8개 컨테이너를 한 번에 관리하기 위해 **Docker Compose**를 사용합니다:

```
# docker-compose.yml
version: '3.8'

services:
  ai-pdm:
    image: shredder-ai-pdm:v1.0
    runtime: nvidia
    restart: always
    volumes:
      - sensor_data:/data
    depends_on:
      - timescaledb

  ai-safety:
    image: shredder-ai-safety:v1.0
    runtime: nvidia
    restart: always
    volumes:
      - sensor_data:/data

  ai-quality:
    image: shredder-ai-quality:v1.0
    restart: always

  timescaledb:
    image: timescale/timescaledb:latest-pg15
```

```
restart: always
volumes:
  - db_data:/var/lib/postgresql/data

mqtt-broker:
  image: eclipse-mosquitto:2
  restart: always
  ports:
    - "1883:1883"

volumes:
  sensor_data:
  db_data:
```

전체 시스템 시작/정지:

```
# 전체 시작 (8개 컨테이너 한 번에)
docker compose up -d

# 전체 정지
docker compose down

# 특정 서비스만 재시작
docker compose restart ai-pdm

# 상태 확인
docker compose ps
```

5.5 Docker와 가상머신(VM)의 차이

비교	가상머신 (VM)	Docker 컨테이너
구조	OS 전체 포함	OS 커널 공유 (경량)
시작 시간	수십 초~수 분	1~3초
메모리	수 GB	수십~수백 MB
디스크	수 GB~수십 GB	수십~수백 MB
성능	오버헤드 있음	네이티브에 가까움
GPU 접근	복잡	NVIDIA Container Runtime으로 간단

Edge AI 장비에서는 자원이 제한적이므로, 무거운 VM이 아닌 경량 Docker 컨테이너가 필수입니다.

5.6 Docker 사용 시 주의사항

주의사항	대책
컨테이너 중단 시 데이터 소실	Volume 마운트로 데이터를 호스트에 영구 저장

주의사항	대책
GPU 접근	NVIDIA Container Runtime 설치 필수 (Jetson은 JetPack에 포함)
네트워크 지연	컨테이너 간 통신은 host 네트워크 모드 사용 시 오버헤드 최소화
이미지 크기	multi-stage build로 최적화 (불필요한 개발 도구 제거)
보안	컨테이너별 최소 권한 부여, 이미지 취약점 스캔

참고 자료

- Edge_AI_슈레더_제안서.md (본 프로젝트 제안서)
- 슈레더_완전가이드_한국기계중심.md (슈레더 기초 학습 자료)
- IMI Sensors: www.pcb.com
- Optris: www.optris.com
- LEM: www.lem.com
- Dräger: www.draeger.com
- Docker Documentation: docs.docker.com
- OPC Foundation: opcfoundation.org
- EtherCAT Technology Group: www.ethercat.org

본 문서는 2026년 3월 26일 기준으로 작성되었습니다.